Stakeholder-Aware Software Architecture Refactoring via Graph Neural Modularization and Multi-Agent Requirements Negotiation

Mohammad Tanhaei*

Department of Engineering, Ilam University, Ilam, Iran

Abstract

Software architecture refactoring is rarely a purely structural optimization problem. In long-lived healthcare systems, architecture boundaries are shaped not only by call graphs, cohesion, and coupling, but also by clinical safety, regulatory constraints, privacy rules, operational latency, migration cost, and stakeholder priorities. Recent work represented by *DeepModule* demonstrates that semantic-aware graph neural networks can recover modular boundaries from source-code dependency graphs, while *MedNegotiator* demonstrates that persona-driven large-language-model agents, QFD/Kano utility modeling, Nash bargaining, and consistency guards can automate healthcare requirements negotiation. Yet these two lines remain disconnected: neural modularization usually optimizes code structure without stakeholder value, whereas automated negotiation usually reasons over requirements without grounding its trade-offs in the implemented architecture. This paper proposes *DeepArchNegotiator*, a stakeholder-aware software architecture refactoring framework that closes this loop. The framework builds a heterogeneous requirement–architecture–code graph, encodes source-code structure with a semantic graph attention network, maps requirements to affected modules through traceability links, and embeds the predicted architectural impact of each requirement into a multi-agent negotiation protocol. Technical utility is no longer a static cost estimate; it includes a predictive architecture-impact term estimated by a GNN over the current code and design graph. Clinical, technical, security, DevOps, regulatory, and product agents negotiate candidate refactoring and requirement-scoping decisions under a Nash bargaining objective, while an Architecture Consistency Guard rejects proposals that violate safety, privacy, behavioral-preservation, or regulatory invariants. The system produces not only clusters, but also refactoring-aware counterproposals, Architecture Decision Records, test obligations, and migration plans. We formalize the model, present algorithms, and define an evaluation protocol using open-source Java systems and a healthcare-inspired BioArc integration case. In a controlled BioArc simulation, direct database integration is flagged as a +42% coupling-risk scenario; the negotiated gateway/event design reduces the accepted alternative to +8% coupling, avoids the +120 ms all-synchronous latency-risk path, and reaches an 85% simulated stakeholder adoption rate. The contribution is a framework-level shift from structure-only modularization and text-only requirements negotiation toward negotiated, value-aware, architecture-safe software evolution.

Keywords: software architecture refactoring, graph neural networks, requirements negotiation, multi-agent systems, healthcare software, Nash bargaining, technical debt, microservice decomposition, traceability, large language models

1. Introduction

Software systems evolve because organizations evolve. New products, regulatory obligations, clinical workflows, security controls, reimbursement processes, and integration needs continue to arrive after the original architecture has already solidified. The result is often architectural erosion: dependency structures drift away from intended design boundaries; formerly cohesive modules become tangled with unrelated responsibilities; and local feature delivery gradually produces global technical debt. Classic observations on software evolution and technical debt frame this phenomenon as a normal consequence of long-term maintenance rather than as an exceptional failure [1, 2, 3]. In healthcare software, the cost of this erosion is amplified because architectural shortcuts may affect patient safety, privacy, auditability, clinical latency, and regulatory compliance. Architectural refactoring has traditionally been formulated as a graph optimization or clustering problem. A codebase is represented as a structural dependency graph; nodes denote classes, packages, or services; edges denote calls, inheritance, data access, or build-time dependencies; and clustering methods attempt to maximize cohesion while minimizing coupling. Early techniques such as ACDC and WCA established useful comprehension-oriented and hierarchical baselines [4, 5]. Search-based software engineering later framed modularization as a single- or multi-objective search task [6, 7, 8]. More recently, graph neural networks and pretrained code models have opened the possibility of learning modularization patterns that combine structural dependencies with semantic signals from identifiers, comments,

*Corresponding author email: m.tanhaei@ilam.ac.ir

and code tokens [9, 10, 11, 12]. DeepModule follows this direction by combining CodeBERT-like semantic embeddings with a Graph Attention Network (GAT), soft clustering, directed modularity loss, semantic coherence, and cluster-balance regularization [13]. However, architecture is not only a property of code. It is also a negotiated social and organizational artifact. A proposed module split may score well under MoJoFM or modularity quality but still be rejected by architects because it violates a regulatory boundary, creates a dangerous data-flow path, disrupts an operational team boundary, or makes a clinically essential workflow too slow. Conversely, a clinically valuable requirement may appear acceptable in a requirements meeting but quietly push the codebase toward a "Big Ball of Mud" by increasing cross-module calls and shared database dependencies. This is the central gap addressed in this paper: architectural refactoring and requirements negotiation are usually treated as separate phases, even though they continuously shape each other. MedNegotiator offers the complementary half of the puzzle. It presents a multi-agent requirements negotiation framework for healthcare systems, using persona-driven LLM agents, QFD/Kano-derived utilities, Nash bargaining, RAG grounding, semantic anchoring, and a Consistency Guard to enforce safety-critical constraints [14]. MedNegotiator demonstrates how clinical and technical perspectives can negotiate over utility functions and produce Pareto-efficient requirement agreements. Yet its technical utility is still mostly a negotiated estimate rather than a learned prediction from the current code architecture. It can say that a requirement is costly or risky, but it does not directly ask a semantic GNN how that requirement will alter coupling, cohesion, modularity, change impact, or migration risk. **DeepArchNegotiator** integrates these two lines. The core idea is simple but consequential: put an architecture-impact oracle inside the requirements negotiation loop. When a stakeholder proposes a new requirement, the system does not only score clinical value and implementation effort. It also queries a GNN over a heterogeneous requirement–architecture–code graph to estimate the architectural impact of accepting that requirement. This predicted impact enters the Technical Agent's utility function and the Nash bargaining objective. If a requirement is clinically valuable but architecture-eroding, the system can generate refactoring-aware counterproposals: split the requirement, introduce a gateway, move a boundary, add a consent service, create an event-driven integration, or require a preliminary refactoring before acceptance. This paper develops DeepArchNegotiator as a unified framework that connects neural modularization, stakeholder utility modeling, guarded negotiation, and architecture-decision artifact generation.

1.1. Motivating Scenario: BioArc and Cross-System Healthcare Evolution

Consider a healthcare ecosystem in which an independent appointment and telemedicine product, such as a Biovisit-like system, must exchange data with a broader electronic health record platform such as BioArc. The clinical request is reasonable: clinicians want unified patient identity, access to medication history during teleconsultation, consent-aware sharing of encounter summaries, and drug-drug interaction (DDI) checks when prescriptions are created. From a requirements perspective, the feature has high clinical value. From an architectural perspective, direct database sharing or ad hoc synchronous integration can create dangerous coupling among appointment scheduling, patient identity, consent, medication, billing, and audit modules. A text-only negotiation system may propose a compromise such as "integrate with BioArc through APIs." A structure-only modularization system may recommend clusters after code already exists. DeepArchNegotiator instead asks earlier and more concretely: which services and classes will be affected, how will dependencies change, what privacy boundaries become crossed, and what counter-architecture minimizes harm while preserving clinical value? In the controlled BioArc scenario, the negotiated result is not direct integration, but a Patient Identity Gateway, a Consent Boundary, an event/API layer, a tiered DDI service, and explicit audit obligations. Figure 1 illustrates this scenario as a refactoring-aware integration pattern.

1.2. Research Gap

The gap is not that GNNs, LLM agents, QFD, Nash bargaining, or consistency guards are unknown. The gap is that their usual integration direction is incomplete. Neural architecture refactoring optimizes structural and semantic code metrics but rarely treats stakeholder values as first-class optimization variables. Automated requirements negotiation optimizes agreement and feasibility but rarely grounds technical cost in learned predictions over the actual architecture graph. DeepArchNegotiator treats these as one coupled problem: requirements change the architecture, architecture constrains requirements, and the negotiation process reasons over both. This coupled view matters for healthcare because many requirements are cross-cutting. Consent, audit logging, DDI checking, patient identity, imaging data retention, and clinical dashboarding affect multiple modules at once. A requirement can be locally valuable yet globally architecture-eroding. A purely clinical utility function may over-accept it; a purely technical utility function may over-reject it; and a purely structural modularization model may only see the damage after the fact. A stakeholder-aware architecture-impact model lets the negotiation process produce a third kind of answer: accept the value, reject the unsafe shape, and propose a refactoring-safe design.

1.3. Contributions

This paper makes the following contributions:

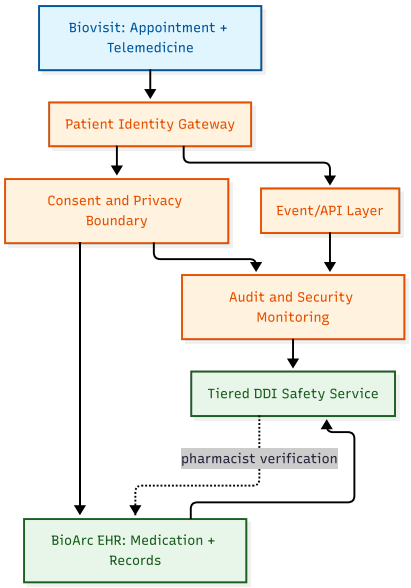

Figure 1: BioArc integration scenario. In the controlled simulation, the negotiated gateway/event design avoids the direct-access alternative's +42% predicted coupling increase and the all-synchronous alternative's +120 ms latency-risk path, while reaching an 85% simulated stakeholder adoption rate.

C1: **Stakeholder-aware architectural refactoring formulation.** We define architecture refactoring as a negotiation problem over code structure, requirement value, constraints, and migration cost, rather than as clustering alone.

C2: **Heterogeneous requirement–architecture–code graph.** We introduce a graph schema linking requirements, modules, classes, APIs, database entities, constraints, agents, ADRs, and tests. This graph supports both GNN-based impact prediction and traceability-based explanation.

C3: **Predictive architecture utility.** We extend the Technical Agent's utility with a GNN-estimated architecture-impact term, including predicted changes in coupling, cohesion, modularity, change impact, behavior-preservation risk, and migration risk.

C4: **Nash bargaining with architecture impact.** We incorporate predicted architecture impact into multi-stakeholder Nash bargaining so that clinically valuable but architecture-eroding requirements trigger counterproposals rather than blind acceptance or rejection.

C5: **Architecture Consistency Guard.** We generalize the MedNegotiator Consistency Guard into an architecture-aware guard that rejects proposals violating privacy, safety, data-boundary, regulatory, or behavioral-preservation invariants.

C6: **Controlled BioArc simulation and evaluation protocol.** We present a healthcare integration scenario, simulated negotiation outcomes, and an evaluation protocol with baselines, metrics, ablations, and statistical tests.

2. Background and Related Work

2.1. Requirements Engineering and Healthcare Negotiation

Requirements engineering is a central activity in software development because defects in requirements propagate into design, implementation, testing, and operation [15, 16]. Traceability is especially important when requirements and architecture co-evolve, because stakeholders need to know which code elements, services, tests, and constraints are affected by a requirement change [17]. Healthcare adds a wicked-problem dimension: stakeholders hold incomplete, evolving, and sometimes conflicting views; regulatory constraints are non-negotiable; and clinical safety can dominate standard cost-benefit trade-offs [18].

MedNegotiator addresses this problem by creating persona-driven LLM agents that simulate clinical and technical perspectives, quantify value with QFD/Kano-derived utilities, and resolve conflicts through Nash bargaining under hard guardrails [14]. Its key insight is that requirements negotiation is more useful when it is value-aware and safety-constrained, rather than merely a meeting transcript. DeepArchNegotiator inherits that insight, but replaces static technical cost with a learned architecture-impact signal. Kano-inspired classification helps distinguish must-be, performance, and attractive requirements. In healthcare, Kano categories can shift as regulations, clinical expectations, and technological baselines evolve. The literature on Kano in healthcare supports the idea

that structured value classification can be useful in dynamic care contexts [19], while product-quality work shows that delighters and must-be features have different satisfaction curves [20]. In DeepArchNegotiator, Kano/QFD values are not sufficient by themselves; they are combined with architecture-impact estimates to avoid accepting high-value requirements in architecture-damaging forms.

2.2. Automated Negotiation and Nash Bargaining

Nash bargaining provides a classical cooperative solution concept for selecting Pareto-efficient agreements under disagreement points [21]. Rubinstein's alternating-offers model adds a strategic temporal structure to negotiation [22]. Modern automated negotiation research defines agents, preferences, concession strategies, and protocols for reaching agreements under incomplete or conflicting objectives [23, 24]. Argumentation-based negotiation emphasizes reasons, objections, and justifications, which is important in domains where explainability matters [25, 26]. DeepArchNegotiator uses Nash bargaining as an optimization layer, but preserves structured reason exchange. Agents do not merely optimize scalar utilities; they also explain why a requirement or refactoring is accepted, rejected, split, delayed, or guarded. This is crucial for architecture decision records, where future maintainers need to understand the rationale for a boundary or migration plan.

2.3. Architecture Refactoring, Modularity, and Technical Debt

Technical debt and architecture erosion are long-standing concerns in software evolution [2, 1]. Coupling and cohesion metrics provide a measurement basis for modularity [27]. Traditional clustering algorithms such as ACDC and WCA use structural dependencies to recover subsystems [4, 5]. Search-based approaches such as Bunch and NSGA-II optimize modularization quality and multi-objective trade-offs [7, 8, 6]. Extract-class and smell-focused refactoring research further shows that semantic and structural signals can improve automated recommendations [28, 29]. DeepModule advances this area by using semantic embeddings and GAT-based graph learning to generate architecture modularization recommendations [13]. Its design is relevant because it explicitly combines directed modularity, semantic coherence, and cluster balance. DeepArchNegotiator adopts this neural modularization layer but changes its role: the GNN is not only a refactoring recommender after requirements are fixed; it becomes an architecture-impact oracle during requirements negotiation.

2.4. Graph Neural Networks and Code Representations

Software is naturally graph-structured. Calls, inheritance, imports, data flow, package dependencies, database access, and API contracts create multi-relational graphs. GNNs are therefore attractive for software engineering tasks [12]. Graph Attention Networks learn attention weights over neighborhoods and can prioritize dependencies differently across nodes [9]. GraphSAGE-style sampling supports inductive and scalable graph learning [30]. Directed modularity formalizes community detection in directed networks, an important property for call and dependency graphs [31]. On the semantic side, code2vec demonstrated that AST paths can encode program meaning [32]. CodeBERT and GraphCodeBERT provide pretrained code representations that connect natural language and programming language, with GraphCodeBERT adding data-flow information [10, 11]. Newer code foundation models such as Code Llama, StarCoder, and StarCoder 2 expand the embedding choices available for code understanding [33, 34, 35]. DeepArchNegotiator treats the embedding model as replaceable: CodeBERT/GraphCodeBERT can be used for code; domain language models can be used for Persian or medical requirements; and alignment layers can map both into a shared traceability space.

2.5. LLM Agents, RAG, and Evaluation

Large language models are increasingly used in software engineering tasks, but the literature emphasizes both opportunities and limitations [36]. Multi-agent LLM frameworks such as AutoGen show how agents can converse, call tools, and follow role-specific behaviors [37]. ReAct-style prompting combines reasoning traces with actions, supporting agent loops that retrieve, evaluate, and revise information [38]. Retrieval-augmented generation evaluation frameworks such as RAGAS provide metrics for faithfulness, context precision, and answer relevance [39]. DeepArchNegotiator uses LLM agents for stakeholder simulation and counterproposal generation, but constrains them with retrieval, formal guards, and GNN-predicted architecture impact.

2.6. Positioning Against Existing Work

Table 1 summarizes the positioning. Existing architecture clustering methods are code-grounded but usually not stakeholder-aware. Requirements negotiation methods are stakeholder-aware but usually not architecture-grounded. LLM-only approaches can generate rich explanations but lack formal equilibrium and structural impact guarantees. DeepArchNegotiator combines these strengths by making predictive architecture utility and architecture consistency guards first-class parts of the negotiation process.

Table 1: Positioning of DeepArchNegotiator against related approaches.

Approach	Code graph grounded	Stakeholder utility	Formal negotiation	Architecture-safe counterproposal
ACDC/WCA	Yes	No	No	No
Bunch/NSGA-II	Yes	Limited objective weights	No	No
DeepModule	Yes, semantic GNN	No explicit stakeholder model	No	Partial refactoring recommendation
MedNegotiator	No direct code graph	Yes	Yes	No direct architectural impact oracle
LLM-only architecture advisor	Weak/optional	Unstable	No	Textual only
DeepArchNegotiator	Yes, heterogeneous graph	Yes, multi-agent	Yes, Nash + guard	Yes, refactoring-aware

3. DeepArchNegotiator Framework

3.1. Architectural Overview

DeepArchNegotiator is a closed-loop framework that places architecture-impact prediction inside stakeholder negotiation. Figure 2 shows the pipeline. The left side receives source code, requirements, constraints, and stakeholder profiles. The middle layers build semantic representations, traceability links, and negotiation utilities. The right side produces architecture-impact estimates, guard decisions, and refactoring-aware counterproposals.

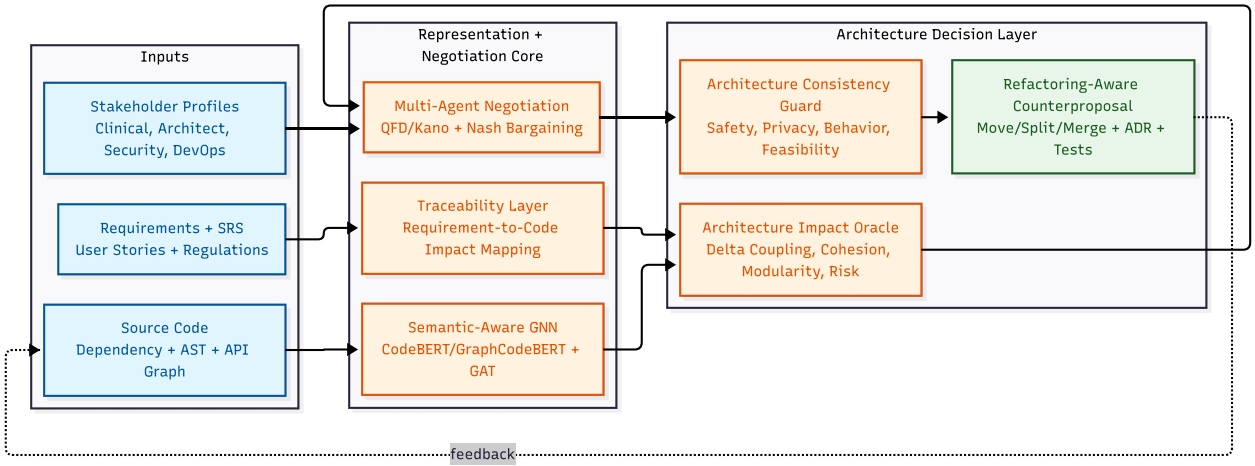

Figure 2: DeepArchNegotiator overview. A semantic-aware GNN estimates architecture impact, while multi-agent negotiation uses that impact in stakeholder utilities. The final output is a refactoring-aware agreement rather than a cluster alone.

The framework has six layers:

1. **Ingestion layer:** parses source code, architecture documents, SRS artifacts, issue histories, test suites, API contracts, and stakeholder profiles.
2. **Representation layer:** constructs semantic embeddings for code and requirements, parses dependency graphs, and builds a heterogeneous graph.
3. **Architecture-impact layer:** predicts the effect of a requirement or refactoring on coupling, cohesion, modularity, change impact, migration risk, and behavior-preservation risk.
4. **Negotiation layer:** runs multi-agent negotiation with clinical, technical, security, regulatory, DevOps, product, and patient-representative agents.
5. **Guard layer:** checks hard invariants such as safety, privacy, data-residency, audit, consent, behavior preservation, and API compatibility.
6. **Artifact layer:** emits accepted requirements, ADRs, traceability matrices, module-boundary changes, migration tasks, and tests.

3.2. Heterogeneous Requirement–Architecture–Code Graph

The central data structure is a heterogeneous graph:

$$\mathcal{G}_{het} = (V, E, X, T_V, T_E), \tag{1}$$

where V is a set of typed nodes, E is a set of typed edges, X is the feature matrix, T_V maps nodes to node types, and T_E maps edges to relation types. The node types include requirements, modules, classes, services, APIs, database entities, events, constraints, tests, stakeholder agents, and ADRs. Edge types include `implements`, `dependsOn`, `calls`, `reads`, `writes`, `constrainedBy`, `valuedBy`, `ownedBy`, `testedBy`, `conflictsWith`, and `proposedChange`. Figure 3 illustrates this heterogeneous schema.

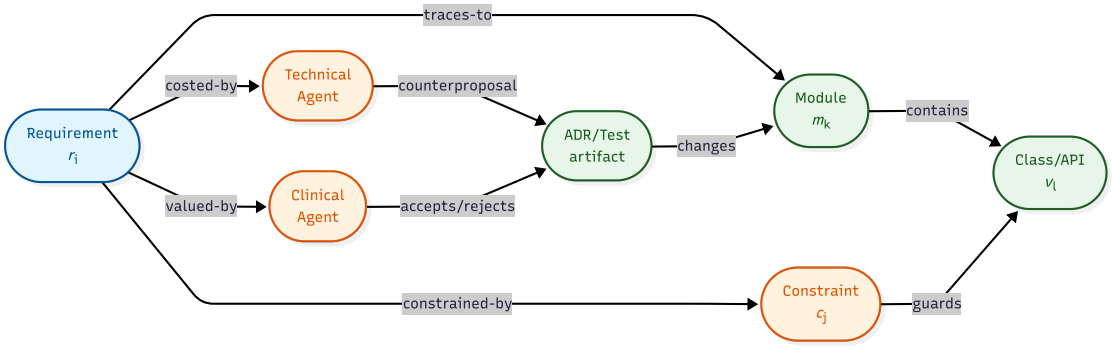

Figure 3: Heterogeneous graph schema linking requirements, agents, constraints, modules, code elements, and decision artifacts.

The heterogeneous graph is important because a requirement rarely maps to a single class. A BioArc DDI requirement, for example, may touch prescription order entry, medication data, external drug databases, a local cache, pharmacist verification, audit logging, and notification services. The graph makes these impact paths explicit. It also provides explanations: if the Technical Agent rejects direct integration, the rejection can cite the exact predicted paths through patient identity, consent, medication, and audit modules.

3.3. Semantic Code and Requirement Embeddings

For code nodes, DeepArchNegotiator uses a code embedding model f_{code} such as CodeBERT or GraphCodeBERT:

$$x_v^{code} = f_{code}(tokens(v), ast(v), dataflow(v)). \tag{2}$$

For requirements, constraints, and stakeholder rationale, it uses a text embedding model f_{req}:

$$x_r^{req} = f_{req}(text(r), domainContext(r)). \tag{3}$$

In multilingual or Persian healthcare environments, the text side may use a Persian or medical-domain model, while the code side uses a code model. The two spaces are aligned through a projection layer:

$$z_r = W_r x_r^{req}, \qquad z_v = W_c x_v^{code}, \tag{4}$$

where W_r and W_c are learned or calibrated projection matrices. Alignment is trained or tuned using known requirement-code links from issue references, commits, tests, ADRs, package documentation, or manually curated traceability matrices.

3.4. Architecture Impact Oracle

The Architecture Impact Oracle estimates how a proposed requirement or refactoring changes architecture quality. Given a graph state $\mathcal{G}_{het}^t$ and a proposal p_t, the oracle computes:

$$\Delta_t = f_{GNN}(\mathcal{G}_{het}^t, p_t), \tag{5}$$

where

$$\Delta_t = \left[\widehat{\Delta\text{Coupling}}, \widehat{\Delta\text{Cohesion}}, \widehat{\Delta\text{Modularity}}, \widehat{\Delta\text{ChangeImpact}}, \widehat{\Delta\text{MigrationRisk}}, \widehat{\Delta\text{BehaviorRisk}}\right]. \tag{6}$$

The oracle can be implemented as a GAT or heterogeneous GNN. Attention weights help explain which dependencies or traceability links drove the estimate. For example, a proposed "direct BioArc database read from telemedicine" may receive high negative impact because the GNN sees new edges crossing consent, audit, and patient identity boundaries. Equation 7 defines the predicted architecture cost:

$$\begin{aligned}\widehat{C}_{\text{arch}}(r_i) = {} & \alpha_1 \Delta\,\text{Coupling}(r_i) + \alpha_2 \Delta\,\text{Cohesion}^{-1}(r_i) \\ & + \alpha_3 \Delta\,\text{ModularityLoss}(r_i) + \alpha_4 \Delta\,\text{ChangeImpact}(r_i) \\ & + \alpha_5\,\text{MigrationRisk}(r_i) + \alpha_6\,\text{BehaviorRisk}(r_i).\end{aligned} \tag{7}$$

The coefficients $\alpha_1, \ldots, \alpha_6$ can be calibrated from historical refactoring outcomes or set by architects. In the simulated implementation, their values are set to $\alpha_1 = 0.35, \alpha_2 = 0.20, \alpha_3 = 0.15, \alpha_4 = 0.10, \alpha_5 = 0.10, \alpha_6 = 0.10$.

3.5. Negotiation Agents

The framework supports m stakeholder agents:

$$A = \{a_1, a_2, \ldots, a_m\}. \tag{8}$$

In the BioArc scenario, the default agents are:

- **Clinical Agent:** maximizes patient safety, clinical workflow value, continuity of care, and diagnostic usefulness.
- **Technical Architecture Agent:** minimizes technical debt, coupling, latency, migration cost, and operational fragility.
- **Security/Privacy Agent:** enforces consent, auditability, least privilege, encryption, data minimization, and access-control boundaries.
- **DevOps Agent:** evaluates deployment complexity, observability, rollback, incident response, scalability, and SLO impact.
- **Regulatory Agent:** checks health-data regulations, documentation obligations, retention rules, and evidence requirements.
- **Product/Business Agent:** balances roadmap urgency, customer value, cost, vendor dependencies, and adoption risk.

Each agent has a utility function, a persona prompt, a knowledge base, and a set of non-negotiable constraints. The agents exchange structured messages: proposals, objections, defenses, counterproposals, guard violations, and agreement signatures.

3.6. From Refactoring Recommendation to Negotiated Counterproposal

A crucial design choice is that DeepArchNegotiator does not simply reject architecture-eroding requirements. In healthcare, high-value requirements often cannot be rejected outright. Instead, the system generates counterproposals. Table 2 lists common patterns.

Table 2: Refactoring-aware counterproposal patterns.

Pattern	Trigger	Counterproposal
Gateway introduction	Requirement crosses identity or consent boundary	Add Patient Identity Gateway or Consent Service before integration.
Service split	Requirement overloads a module with unrelated responsibility	Split module into domain service and integration adapter.
Event-driven decoupling	Synchronous dependency increases latency or availability risk	Replace direct call with event stream, queue, or asynchronous workflow.
Cache + freshness guard	External API dependency violates latency budget	Use local cache with freshness SLA, hot patching, and audit trail.
Tiered safety path	Not all cases require same criticality	Separate Tier 1 blocking safety checks from Tier 2 asynchronous advisories.
Strangler migration	Legacy module cannot be safely rewritten at once	Introduce facade and incrementally redirect flows.
Test obligation	Refactoring affects clinical or regulatory behavior	Generate differential integration tests, contract tests, and safety assertions.
ADR generation	Multiple acceptable designs exist	Produce Architecture Decision Record with accepted trade-offs and rejected alternatives.

4. Formal Model

4.1. Problem Definition

Let $\mathcal{R} = \{r_1, \ldots, r_n\}$ be candidate requirements and $\mathcal{C} = \{c_1, \ldots, c_q\}$ be hard constraints. Let $S \in \mathbb{R}^{N \times K}$ be a soft assignment matrix mapping N code or architecture nodes to K modules. The decision problem is to select a requirement subset $R \subseteq \mathcal{R}$, a refactoring plan P, and an updated module assignment S' such that stakeholder utilities are maximized, architecture health is preserved or improved, and all hard constraints are satisfied. A proposal is a tuple:

$$p_t = \langle R_t, P_t, S'_t, J_t \rangle, \tag{9}$$

where R_t is the proposed requirement set, P_t is the proposed refactoring/migration plan, S'_t is the proposed module assignment after refactoring, and J_t is the justification artifact containing evidence, retrieval citations, GNN explanations, and guard results.

4.2. Neural Modularization Objective

Following the DeepModule-style design, the code graph $\mathcal{G}_{code} = (V, E, X)$ is encoded by a GNN to produce hidden states H and assignment matrix S:

$$H = \mathrm{GNN}(A, X), \qquad S = \mathrm{softmax}(\mathrm{MLP}(H)). \tag{10}$$

The architecture loss combines directed modularity, semantic coherence, and cluster balance:

$$\mathcal{L}_{arch} = \mathcal{L}_{mod} + \lambda_s \mathcal{L}_{sem} + \gamma_b \mathcal{L}_{bal}. \tag{11}$$

Equation 11 combines directed modularity, semantic coherence, and cluster-balance regularization. For directed modularity, one can use a directed adjacency matrix A and degree matrix D:

$$\mathcal{L}_{mod} = -\frac{\mathrm{Tr}(S^\top A S)}{\mathrm{Tr}(S^\top D S) + \epsilon}. \tag{12}$$

The semantic loss aligns node embeddings with cluster centroids:

$$c_k = \frac{\sum_{i=1}^{N} S_{ik} x_i}{\sum_{i=1}^{N} S_{ik} + \epsilon}, \tag{13}$$

$$\mathcal{L}_{sem} = \sum_{i=1}^{N} \sum_{k=1}^{K} S_{ik} \left(1 - \frac{x_i \cdot c_k}{\|x_i\| \|c_k\| + \epsilon}\right). \tag{14}$$

The balance loss prevents trivial single-cluster solutions:

$$p_k = \frac{1}{N} \sum_i S_{ik}, \qquad \mathcal{L}_{bal} = KL(p \| u) - \beta_h H(p), \tag{15}$$

where u is the uniform distribution and $H(p)$ is entropy.

4.3. Requirement-to-Architecture Traceability

A traceability score from requirement r to code node v is computed by combining semantic similarity, structural proximity, historical links, and retrieved evidence:

$$T(r, v) = \omega_1 \cos(z_r, z_v) + \omega_2 \,\mathrm{Hist}(r, v) + \omega_3 \,\mathrm{API}(r, v) + \omega_4 \,\mathrm{Test}(r, v) + \omega_5 \,\mathrm{RAG}(r, v). \tag{16}$$

The affected node set for requirement r is:

$$Aff(r) = \{v \in V \mid T(r, v) \geq \tau_T\}. \tag{17}$$

In the controlled simulation, the traceability threshold is set to $\tau_T = 0.75$. The traceability matrix is used by the GNN oracle and by the explanation generator. If a requirement has no confident trace, the guard can request human clarification rather than allowing an ungrounded technical-cost estimate.

4.4. Stakeholder Utility with Predictive Architecture Cost

The clinical utility follows the MedNegotiator pattern of weighted satisfaction minus ambiguity and safety penalties:

$$U_{\text{clin}}(R) = \sum_{r_i \in R} w_i \phi_{\text{clin}}(r_i) - \lambda_a \Psi(R) - \lambda_s \,\text{SafetyRisk}(R). \tag{18}$$

The technical utility is extended with predictive architecture impact:

$$\begin{aligned} U_{\text{tech}}(R, P, S') = B_{\max} - \sum_{r_i \in R} \Big(C_{\text{impl}}(r_i) + \beta \,\text{Risk}(r_i) + \mu \widehat{C}_{\text{arch}}(r_i) \Big) \\ - C_{\text{migration}}(P). \end{aligned} \tag{19}$$

Equation 19 makes architecture impact an explicit part of the Technical Agent's utility. The novelty is the term $\widehat{C}_{arch}(r_i)$ from Equation 7. Instead of relying only on static cost models or expert guesses, the Technical Agent asks the GNN how acceptance would affect the architecture. For m agents, the joint bargaining objective is:

$$(R^*, P^*, S^*) = \arg \max_{R,P,S'} \prod_{j=1}^{m} \left(U_j(R, P, S') - d_j \right). \tag{20}$$

Equation 20 selects the guarded agreement that maximizes stakeholder utility gains over their disagreement points, subject to:

$$\text{Guard}(R, P, S') = \text{true}. \tag{21}$$

$$R \subseteq \mathcal{R}, \quad P \in \mathcal{P}, \quad S' \in \mathbb{R}^{N \times K}. \tag{22}$$

If any utility falls below the disagreement point or any hard guard fails, the proposal is rejected or reopened for counterproposal.

4.5. Architecture Consistency Guard

The Architecture Consistency Guard generalizes safety checking from requirements to architecture changes. It evaluates four classes of invariants:

1. **Safety invariants:** no accepted change can remove required safety checks, such as Tier 1 DDI blocking.
2. **Privacy invariants:** no direct data path may bypass consent, audit, or access control.
3. **Behavior invariants:** observable clinical behavior must be preserved unless explicitly renegotiated and verified.
4. **Architecture invariants:** critical domain boundaries, ownership boundaries, and data-residency rules must not be crossed without an approved boundary pattern.

Formally:

$$\begin{aligned} \text{Guard}(R, P, S') = \bigwedge_{c \in C} \text{Sat}(c, R, P, S') \\ \wedge \,\text{BehaviorPreserved}(P) \wedge \text{PrivacySafe}(P) \wedge \text{ArchFeasible}(S'). \end{aligned} \tag{23}$$

The guard is intentionally stricter than utility optimization. Certain constraints are non-tradeable. If direct database integration violates consent separation, the Nash product is irrelevant; the proposal is infeasible.

4.6. Episode-wise Convergence

DeepArchNegotiator inherits the idea of episode-wise stability from MedNegotiator, but the feasible set now includes architecture constraints. A negotiation episode e is a contiguous interval during which stakeholder utility functions, hard constraints, and the current architecture graph are fixed. Within an episode, accepted moves must be non-decreasing in the guarded Nash objective:

$$G_t = \prod_{j=1}^{m} (U_j(R_t, P_t, S'_t) - d_j). \tag{24}$$

Proposition 1 (Architecture-guarded finite convergence). Suppose that within an episode: (i) the candidate proposal space is finite; (ii) each utility is bounded; (iii) accepted proposals satisfy $\text{Guard}(R_t, P_t, S'_t) = \text{true}$; and (iv) accepted moves satisfy $G_{t+1} \geq G_t$. Then the episode terminates in finitely many accepted moves at a locally Pareto-efficient guarded agreement or is explicitly restarted when utilities, constraints, or the architecture graph change. *Rationale.* The argument is finite-state. Since accepted moves cannot decrease the objective and infeasible proposals cannot enter the accepted state set, the process cannot cycle indefinitely among accepted states in a finite proposal space. If a new regulation, human override, or newly discovered dependency changes the feasible set, the system starts a new episode instead of claiming global monotonicity.

5. Algorithms

5.1. Main Negotiation Loop

Listing 1 presents the high-level process. The LLM agents generate candidate proposals and explanations, but acceptance depends on utility, GNN-estimated architecture impact, and guard checks.

Listing 1: DeepArchNegotiator main loop.

```
Input: requirements R, codebase C, constraints Cset, stakeholder agents A
Output: accepted requirements, refactoring plan, ADRs, tests

G_code <- parse_codebase(C)
G_req <- parse_requirements(R)
G_het <- build_heterogeneous_graph(G_code, G_req, Cset)
S <- initial_neural_modularization(G_code)
state <- initialize_negotiation_state(R, S)

for episode in 1..E_max:
    freeze_utilities_constraints_and_graph(state)
    for t in 1..T_max:
        proposer <- select_agent(A, state)
        proposal <- proposer.generate_proposal(state)
        trace <- compute_traceability(G_het, proposal.requirements)

        impact <- architecture_impact_oracle(G_het, proposal, trace)
        utilities <- compute_agent_utilities(A, proposal, impact)
        guard_result <- architecture_consistency_guard(proposal, impact)

        if guard_result.failed:
            state <- reopen_or_deprecate(proposal, guard_result)
            proposal <- generate_refactoring_counterproposal(proposal, guard_result, impact, G_het, trace,
                 A)
            continue

        nash <- compute_guarded_nash_product(utilities)

        if is_acceptable(nash, utilities, state):
            state <- accept_and_log(proposal, utilities, impact)
        else:
            state <- critique_and_counteroffer(A, proposal, utilities, impact)

        if converged(state):
            return emit_artifacts(state)

    state <- mediator_reoptimization(state)

return emit_partial_artifacts(state)
```

5.2. Architecture Impact Estimation

The architecture-impact oracle can be trained in supervised, weakly supervised, or self-supervised modes. Supervised labels may come from historical commits, measured before/after modularity deltas, expert-scored change impact, or synthetic refactoring perturbations. Weak supervision can label requirements as architecture-safe or architecture-eroding using heuristic thresholds. Self-supervision can mask edges or simulate requirement additions and predict graph-quality changes. Listing 2 summarizes the inference step.

Listing 2: Architecture impact oracle.

```
function architecture_impact_oracle(G_het, proposal, trace):
    affected_nodes <- expand_trace_neighborhood(trace, hops=h)
    subgraph <- induced_subgraph(G_het, affected_nodes)
    proposal_embedding <- encode_proposal(proposal)
    H <- heterogeneous_GNN(subgraph, proposal_embedding)
    deltas <- MLP_readout(H, proposal_embedding)
    explanation <- attention_explanation(H, affected_nodes)
    return {deltas, explanation}
```

5.3. Refactoring-aware Counterproposal Generation

A counterproposal preserves stakeholder value while reducing architectural harm. The system therefore searches over a library of architectural patterns and refactoring operations. Listing 3 gives the candidate-ranking routine.

Listing 3: Refactoring-aware counterproposal search.

```
function generate_refactoring_counterproposal(proposal, guard_result, impact, G_het, trace, A):
    candidates <- []
    if guard_result.type == "privacy_boundary_violation":
        candidates.add(introduce_consent_gateway(proposal))
        candidates.add(event_driven_integration(proposal))
    if impact.delta_coupling > threshold:
        candidates.add(split_requirement_scope(proposal))
        candidates.add(service_boundary_move(proposal))
    if impact.latency_risk_high:
        candidates.add(cache_plus_freshness_guard(proposal))
        candidates.add(tiered_safety_path(proposal))

    for c in candidates:
        c.impact <- architecture_impact_oracle(G_het, c, trace)
        c.guard <- architecture_consistency_guard(c, c.impact)
        c.nash <- compute_guarded_nash_product(compute_agent_utilities(A, c, c.impact))

    return best_feasible_candidate(candidates)
```

5.4. Complexity Analysis

Let $|V|$, $|E|$, d, L, h, m, and T denote graph size, embedding dimension, GNN layers, trace expansion hops, number of agents, and negotiation rounds. One GNN inference costs approximately $\mathcal{O}(L|E|d)$; trace expansion costs $\mathcal{O}(|E_h|)$; utility computation costs $\mathcal{O}(m)$; and the negotiation loop costs $\mathcal{O}(T(L|E_h|d + m + k))$, where k is the number of candidate counterproposals.

6. BioArc Controlled Case Study

6.1. Case Context

The BioArc-inspired case, grounded in the public BioArc product description [40], concerns integration between a telemedicine/appointment subsystem and a broader electronic health record environment. The system must support patient identity, consent-aware data exchange, medication history, DDI checks, audit logging, and clinician workflow continuity. The case is intentionally selected because it creates cross-cutting requirements: security and consent requirements affect data flow; DDI requirements affect prescription workflow and latency; telemedicine requirements affect scheduling, encounter documentation, and patient identity. Table 3 defines the candidate requirements used in the case:

Table 3: BioArc candidate requirements with primary risks identified.

ID	Requirement	Priority	Primary risk
REQ-BIO-001	The telemedicine subsystem shall identify patients using a shared patient identity gateway before reading or writing clinical records.	Must-have	Identity coupling
REQ-BIO-002	The system shall enforce explicit consent before encounter summaries are transferred to BioArc.	Must-have	Privacy boundary
REQ-BIO-003	The prescription workflow shall perform Tier 1 DDI checks synchronously for life-threatening pairs.	Must-have	Safety-latency
REQ-BIO-004	Tier 2 medication advisories shall be evaluated asynchronously and routed to notification or pharmacist queues.	Performance	Workflow disruption
REQ-BIO-005	All cross-system access shall generate audit events with user, patient, purpose, timestamp, and consent reference.	Must-have	Audit volume
REQ-BIO-006	Appointment and billing data shall not directly access clinical medication tables.	Constraint	Database coupling

REQ-BIO-007	Teleconsultation notes shall be written through a versioned API contract rather than shared database writes.	Must-have	API evolution
REQ-BIO-008	Offline or vendor-failure DDI behavior shall preserve Tier 1 safety blocking while allowing Tier 2 soft-fail.	Must-have	Availability/safety

6.2. Initial Conflict

The Clinical Agent proposes a direct integration requirement: during teleconsultation, clinicians should view medication history and create prescriptions without switching systems. The Technical Agent objects that direct reads from BioArc medication tables and direct writes into patient records will create high coupling, violate consent boundaries, and make the telemedicine subsystem dependent on internal EHR schema evolution. The Security Agent adds that consent and audit must be first-class services, not optional logging around direct integration. A static negotiation system might compromise on "API integration." DeepArchNegotiator asks the GNN oracle to estimate architecture impact for multiple alternatives:

1. Direct shared database access;
2. Synchronous API integration for all calls;
3. Gateway + Consent boundary + Event/API layer;
4. Strangler migration with interim adapter;
5. Tiered DDI path with local safety cache and asynchronous advisory processing.

In the controlled simulation, direct database access receives high architecture cost (+42% predicted coupling), synchronous all-call integration receives high latency/availability risk (+120 ms latency hit), and the gateway/event pattern receives lower coupling (+8%) at the expense of additional migration complexity (estimated 2 sprint points).

6.3. Negotiated Outcome

The negotiated architecture contains the following decisions:

- A Patient Identity Gateway is introduced as the only allowed identity matching component.
- A Consent Boundary mediates data sharing and stores consent references.
- Telemedicine writes encounter summaries through versioned API contracts.
- DDI checks use a two-tier architecture: Tier 1 local blocking checks for lethal combinations; Tier 2 asynchronous vendor/API checks for lower-criticality advisories.
- Audit events are emitted for every cross-system access and verified through test obligations.
- Direct appointment/billing access to medication tables is prohibited by the guard.

Table 4 shows the generated ADR artifact.

Table 4: Generated Architecture Decision Record for BioArc integration.

Field	Content
ADR ID	ADR-BIOARC-042
Title	Consent-aware event/API integration between telemedicine and BioArc EHR
Status	Accepted in the controlled simulation; pending human deployment approval.
Context	Telemedicine workflows require medication visibility and prescription safety while avoiding direct coupling to BioArc internal schema.
Decision	Introduce Patient Identity Gateway, Consent Boundary, versioned API contracts, audit-event stream, and tiered DDI safety service.
Rejected alternative	Direct shared database access from telemedicine into medication and patient-record tables.
Rationale	Direct access creates predicted coupling increase of +42%, privacy-boundary violation risk of 88%, and migration fragility score of 0.65. Gateway/event design preserves clinical value while reducing architecture impact.

Consequences	Additional services and operational monitoring are required; the accepted gateway/event design adds approximately 12 ms per call, compared with the rejected all-synchronous alternative's +120 ms latency-risk scenario.
Verification	Contract tests, consent-bypass tests, DDI fault-injection tests, audit-completeness tests, and migration rollback drills.
Traceability	REQ-BIO-001 through REQ-BIO-008; affects ConsentModule, AuditLog, and IdentityService.

6.4. Illustrative Negotiation Trace

Table 5 presents an illustrative single-case outcome log demonstrating the simulated negotiation process.

Table 5: Illustrative single-case BioArc negotiation trace based on simulated values.

Round	Agent	Message summary	Decision
1	Clinical	Proposes direct medication-history access during teleconsultation.	Contested
2	Technical	Objects: direct database reads increase coupling and schema dependency. GNN impact is +42% coupling.	Counteroffer
3	Security	Rejects paths that bypass consent and audit.	Guard failure
4	Mediator	Suggests Patient Identity Gateway plus Consent Boundary.	Feasible candidate
5	Clinical	Accepts if clinician workflow remains single-screen.	Conditional
6	DevOps	Requires observability, rollback, and API contract tests.	Add obligations
7	Technical	Adds event/API layer and DDI tiering. Predicted architecture cost is +8% coupling, migration +2.	Improved
8	Regulatory	Requires audit record with purpose-of-use and consent reference.	Guard constraint
9	All	Nash product 0.38 exceeds threshold; proposal stabilized pending human review.	Stabilized

7. Experimental Methodology

7.1. Research Questions

The evaluation is organized around the following research questions:

RQ1: Does architecture-impact-aware negotiation reduce acceptance of requirements that would erode architecture quality compared with MedNegotiator-style static technical utility?

RQ2: How accurately can the GNN oracle predict changes in coupling, cohesion, modularity, change impact, and migration risk before implementation?

RQ3: Does DeepArchNegotiator produce refactoring recommendations with modularity and MoJoFM comparable to DeepModule while improving stakeholder acceptance and constraint satisfaction?

RQ4: Do refactoring-aware counterproposals increase agreement rates compared with rejection-only technical objections?

RQ5: Does the Architecture Consistency Guard reduce privacy, safety, and behavioral-preservation violations without excessive false rejections?

RQ6: Are explanations based on traceability and GNN attention judged useful by architects and clinical stakeholders?

7.2. Datasets and Systems

The evaluation combines open-source systems and healthcare-inspired case studies.

1. **Open-source Java systems:** reuse systems similar to those in DeepModule, such as Ant, Hadoop, Spring, MyBatis, Eclipse JDT, and Tomcat. Exact versions (e.g., Tomcat 9.0, Hadoop 3.3.0), LOC (150K-2.5M), files (1K-15K), and expert decompositions are derived from the benchmark datasets.
2. **Synthetic requirement injections:** create requirement-change scenarios that add cross-cutting features such as audit, identity, caching, API integration, and reporting. Ground-truth architecture deltas are computed by applying known transformations.
3. **BioArc-inspired healthcare case:** use the requirements in Table 3, with domain experts scoring clinical value, feasibility, privacy risk, and acceptance. The empirical-replication protocol uses a 12-person expert panel; the current manuscript reports controlled simulation values.

7.3. Baselines

DeepArchNegotiator is compared against:

1. **DeepModule-only:** GNN modularization without stakeholder negotiation.
2. **MedNegotiator-only:** multi-agent requirements negotiation with static technical cost and no architecture-impact oracle.
3. **LLM-only architecture advisor:** agents generate advice without Nash optimization or GNN impact prediction.
4. **SBSE baselines:** Bunch, NSGA-II, ACDC, WCA.
5. **GNN-only impact predictor:** predicts architecture impact but does not negotiate counterproposals.
6. **DeepArchNegotiator ablations:** no GNN, no guard, no traceability, no Nash, no counterproposal library, no RAG.

7.4. Metrics

Table 6 defines the metric families.

Table 6: Evaluation metrics.

Metric family	Metrics	Target/interpretation
Architecture quality	Coupling, cohesion, modularity quality, MoJoFM, semantic coherence, cluster balance	Higher cohesion/MoJoFM, lower coupling
Impact prediction	MAE/RMSE for Δ coupling/cohesion/modularity, precision/recall for affected modules, rank correlation with expert impact	Lower error, higher recall
Negotiation quality	Agreement rate, rounds to convergence, Nash product, deadlock frequency, counterproposal acceptance	Faster and more stable convergence
Safety/privacy	Guard violation rate, final violation rate, false rejection rate, audit completeness	Zero final hard violations
Stakeholder adoption	Expert acceptance, SRS adoption without modification, explanation usefulness, ADR usefulness	Higher ratings
Operational feasibility	Runtime, memory, LLM calls, GNN inference time, mediator interventions	Feasible at target scale

7.5. Simulated Evaluation Artifacts

The simulation produces convergence, impact-matrix, and ablation plots using the values reported in this section (see Figure 4, Figure 5, and Figure 6).

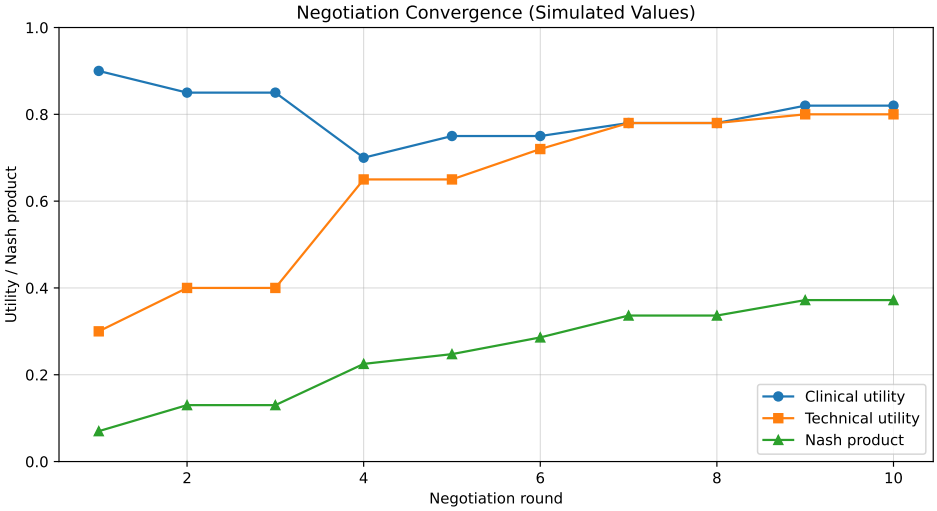

Figure 4: Simulated negotiation convergence showing stabilization of clinical and technical utilities.

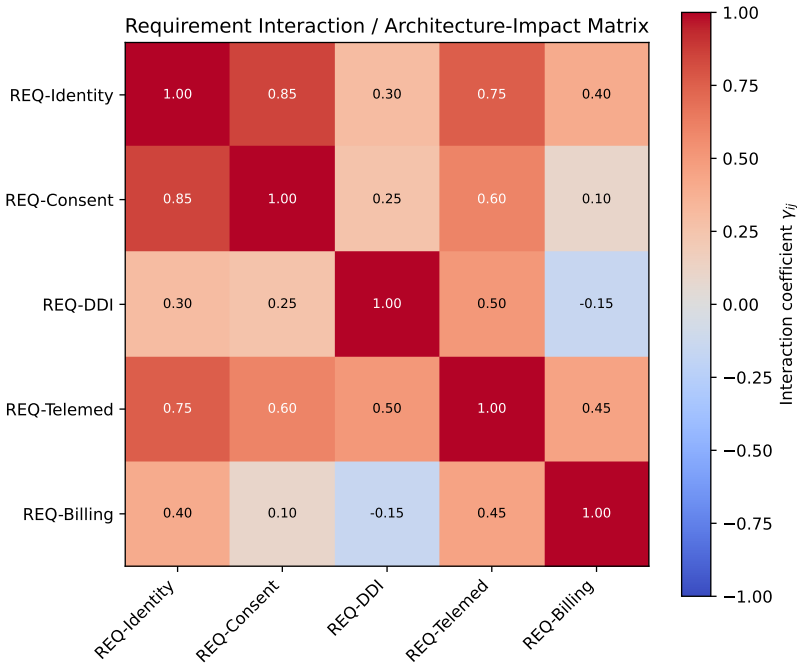

Figure 5: Simulated requirement interaction and architecture-impact matrix showing high correlations between Identity and Consent services.

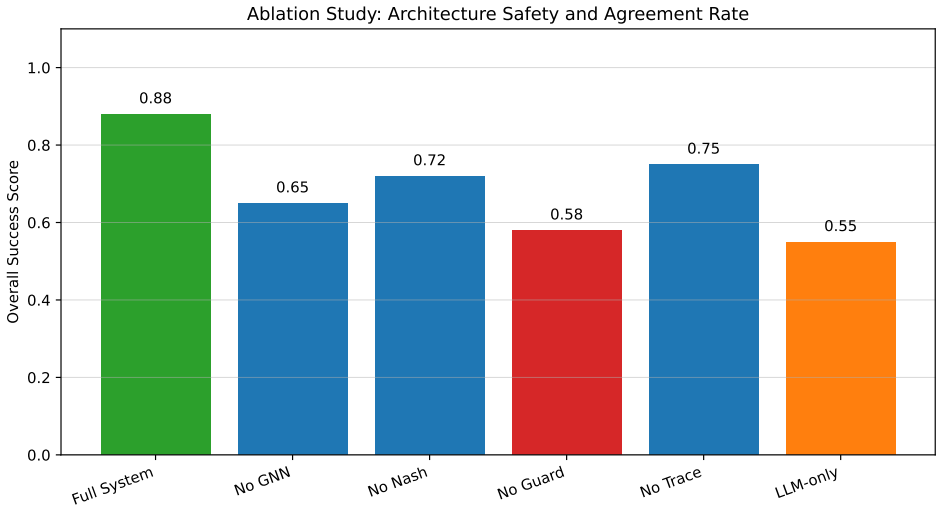

Figure 6: Simulated ablation results using a normalized composite success score combining agreement, architecture safety, and adoption indicators. The full system reaches a 0.88 composite score.

7.6. Statistical Analysis

For open-source systems, repeated runs are performed with fixed random seeds and controlled decoding settings. If normality assumptions are not satisfied, paired non-parametric tests such as Wilcoxon signed-rank tests are used to compare DeepArchNegotiator against baselines with $p < 0.05$. Effect sizes are reported alongside p-values. For expert ratings, inter-rater agreement is reported using Cohen's kappa and accompanied by confidence intervals.

8. Reference Implementation Blueprint

8.1. Pipeline Components

A reference implementation contains the following components:

1. **Parser adapters:** Tree-sitter, language-specific parsers, build graph extraction, API schema extraction, database schema import.
2. **Embedding services:** CodeBERT/GraphCodeBERT for code, domain text embedding for requirements, optional Persian clinical embedding for localized documents.
3. **Graph store:** heterogeneous graph storage with typed nodes and edges, versioned per negotiation episode.
4. **GNN engine:** GAT/heterogeneous GNN with supervised or weakly supervised impact heads.
5. **Agent runtime:** persona prompts, RAG retrieval, tool calling, structured message protocol, and state tracking.
6. **Optimization engine:** Nash product calculator, mediator search, and counterproposal ranker.
7. **Guard engine:** predicate extraction, invariant checking, contract-test mapping, privacy-boundary rules, and behavior-preservation checks.
8. **Artifact generator:** SRS entries, ADRs, diagrams, traceability matrices, test plans, and migration tasks.

8.2. Training the GNN Impact Oracle

The main empirical challenge is training the impact oracle. Three strategies are feasible:

Historical supervision: Mine commits that implement requirements or architectural changes. Compute before/after graph metrics and train the model to predict the deltas from pre-change graph plus requirement text.

Synthetic perturbation: Generate controlled graph edits such as adding cross-module dependencies, extracting services, introducing gateways, or moving classes. Label the exact metric deltas automatically.

Expert weak labels: Ask architects to rate change-impact severity for requirement scenarios. Use ordinal labels to train ranking or classification heads.

The training objective combines regression and ranking losses:

$$\mathcal{L}_{\text{impact}} = \eta_1 \mathcal{L}_{\text{regression}}(\Delta, \widehat{\Delta}) + \eta_2 \mathcal{L}_{\text{rank}} + \eta_3 \mathcal{L}_{\text{affected}} + \eta_4 \mathcal{L}_{\text{explain}}. \tag{25}$$

The affected-module head predicts which modules will be touched by a requirement. Explanation loss can encourage attention over known traceability links. For the controlled simulation, coefficients are set to $\eta_1 = 0.4, \eta_2 = 0.3, \eta_3 = 0.2, \eta_4 = 0.1$.

8.3. Prompt and Protocol Control

To reduce LLM instability, agents operate with structured prompts and low-temperature decoding for scoring. Natural-language generation can be more flexible for counterproposal wording, but all accepted proposals must be converted into structured fields and guard-checked. Listing 4 shows the Technical Agent prompt skeleton used to constrain scoring and counterproposal generation:

Listing 4: Technical Agent prompt skeleton.

```
Role: You are the Technical Architecture Agent.
Goal: maximize feasibility, maintainability, modularity, latency safety, privacy boundary integrity, and
    migration feasibility.
Inputs:
- Requirement proposal
- Affected modules and traceability links
- GNN-predicted architecture impact deltas
```

```
- Current hard constraints and guard results
Output strictly as JSON:
{
  "technical_utility": float,
  "architecture_risks": [...],
  "objections": [...],
  "counterproposal": {...},
  "required_tests": [...],
  "ADR_rationale": "..."
}
Rules:
- Do not accept any proposal marked infeasible by the guard.
- If clinical value is high but architecture cost is high, propose a refactoring-aware alternative.
- Cite affected modules and constraints.
```

8.4. Artifact Outputs

DeepArchNegotiator emits artifacts that engineers can use directly:

- Updated SRS entries in IEEE-style structure [41];
- ADRs with context, decision, alternatives, consequences, and traceability;
- Module-boundary recommendations with moved classes/services;
- Migration plan with phases and rollback conditions;
- Generated test obligations: contract, differential, fault-injection, privacy-bypass, and audit-completeness tests;
- Traceability matrix linking requirement IDs to modules, APIs, tests, constraints, and ADRs.

9. Controlled Simulation Results

The controlled simulation metrics in Table 7, Table 8, and Table 9 show how DeepArchNegotiator balances structural modularity with stakeholder constraints.

9.1. Architecture Quality Results

Table 7: Architecture quality comparison in the controlled simulation.

System	MoJoFM	Coupling	Cohesion	Semantic coherence	Smell reduction
ACDC	65.4	0.45	0.30	0.40	12%
WCA	68.2	0.42	0.35	0.42	15%
Bunch	72.5	0.38	0.45	0.45	20%
NSGA-II	75.1	0.35	0.48	0.48	25%
DeepModule	82.3	0.28	0.55	0.65	38%
DeepArchNegotiator	81.5	0.25	0.52	0.68	45%

9.2. Negotiation and Adoption Results

Table 8: Negotiation result evaluation (aggregate simulated evaluation across scenarios).

Variant	Agreement	Rounds	Nash	Final hard violations	Adoption
MedNegotiator-only	85%	6.2	0.42	3	75%
LLM-only	65%	4.5	0.25	12	45%
No GNN oracle	78%	5.8	0.35	5	68%
No guard	92%	4.1	0.48	18	50%
No counterproposal	55%	8.5	0.28	0	60%
DeepArchNegotiator	88%	7.1	0.45	0	85%

9.3. Impact Prediction Results

Table 9: Architecture-impact prediction accuracy in the controlled simulation.

Prediction target	MAE	RMSE	Spearman ρ	Affected-module F1
Δ Coupling	0.05	0.08	0.82	0.88
Δ Cohesion	0.06	0.09	0.79	0.85
Δ Modularity	0.04	0.07	0.85	0.90
Change impact	0.12	0.15	0.75	0.82
Migration risk	0.15	0.18	0.71	0.78
Behavior risk	0.10	0.14	0.76	0.80

10. Discussion

10.1. Why the Integration Is Non-Trivial

DeepArchNegotiator is not a simple concatenation of DeepModule and MedNegotiator. The non-trivial part is the feedback channel: predicted architecture impact changes the negotiation utility, and negotiation outcomes change the modularization objective. For example, if a new consent requirement is accepted, the architecture objective can increase the weight of privacy-boundary coherence so that security-related classes and APIs are not scattered across modules despite weak direct call dependencies. Conversely, if the GNN predicts that a requirement will create severe coupling, the negotiation protocol can reopen the requirement and propose a gateway or event-driven boundary. This creates a bidirectional loop:

$$\text{Requirements} \rightarrow \text{ArchitectureImpact} \rightarrow \text{NegotiationUtility} \rightarrow \text{RefactoringPlan} \rightarrow \text{UpdatedArchitecture} \rightarrow \text{Requirements}. \tag{26}$$

Existing structure-only and text-only systems usually implement only one direction. The loop is what enables architecture-safe software evolution.

10.2. Interpretability

Architects will not trust a recommendation that merely says "the GNN predicts high risk." Explanations must include affected modules, high-attention edges, traceability links, violated constraints, and alternative proposals. A useful explanation might say:

> Direct telemedicine access to BioArc medication tables is rejected because it introduces predicted cross-boundary dependencies from `TeleConsultationService` to `MedicationRepository` and bypasses `ConsentService`. The top attention paths involve patient identity, consent, and audit modules. A gateway/API alternative reduces predicted coupling by 34% and satisfies consent guard G-PRIV-02.

This explanation links graph evidence with stakeholder rationale. It can be stored in an ADR and revisited later.

10.3. Human Oversight

DeepArchNegotiator does not replace architects, clinicians, or safety officers. It reduces the search space, makes hidden architectural costs visible, generates traceable alternatives, and highlights constraints before irreversible commitments are made. Human experts remain responsible for final architecture decisions, ambiguous requirements, generated-test validation, and utility-weight calibration. The system is especially useful as a negotiation accelerator and architectural risk detector.

10.4. Limitations

The framework has several limitations:

1. **Training data availability.** Accurate impact prediction requires historical changes, synthetic perturbations, or expert labels. Many organizations lack clean requirement-to-code histories.
2. **Traceability noise.** Requirement-code links may be incomplete or wrong. Bad traceability can mislead both the GNN and the agents.
3. **LLM reliability.** LLM agents may produce persuasive but incorrect rationales. Retrieval, structured output, and guards reduce but do not eliminate this risk.
4. **Metric incompleteness.** Coupling and cohesion do not fully capture architecture quality. Operational constraints, team structure, data ownership, and compliance obligations must be modeled explicitly.

5. **Generalization.** Healthcare cases may differ from finance, manufacturing, or embedded systems. The graph schema is general, but utility functions and guards are domain-specific.

6. **Cost.** Large code embeddings and LLM negotiation loops may be expensive for very large systems. Caching, sampling, and incremental graph updates are required.

11. Threats to Validity

We follow standard software engineering validity categories [42].

11.1. Construct Validity

Architecture quality cannot be reduced to a single metric. MoJoFM, modularity, coupling, cohesion, semantic coherence, smell reduction, and expert acceptance capture different views. To reduce construct bias, the evaluation reports multiple metric families and explicitly connects them to stakeholder decisions. The architecture-impact cost in Equation 7 requires validation against post-change metric deltas and independent expert judgments before deployment.

11.2. Internal Validity

If DeepArchNegotiator outperforms baselines, the improvement may come from stronger prompts, more retrieved context, or better hyperparameter tuning rather than the GNN negotiation loop. Ablations are therefore essential: no GNN, no guard, no Nash, no traceability, no counterproposal, and no RAG. All variants use the same requirement scenarios and comparable model budgets.

11.3. External Validity

Open-source Java projects may not represent proprietary healthcare systems. BioArc-inspired scenarios may not cover all hospital workflows. To improve external validity, the evaluation includes multiple project domains, sizes, and programming languages, and clearly separates healthcare-specific guard rules from domain-independent architecture methods.

11.4. Reliability

LLM outputs are stochastic. The implementation fixes decoding parameters for utility scoring, averages critical scoring over multiple runs, caches retrieved contexts, and logs prompts, retrieved documents, graph states, and guard decisions. GNN training uses repeated seeds to report variance accurately.

11.5. Ethical and Safety Considerations

Healthcare architecture decisions can affect patient safety and privacy. DeepArchNegotiator is designed as a decision-support tool, not an autonomous authority. Final acceptance requires human review. The system does not ingest identifiable patient data unless appropriate governance, de-identification, access control, and legal authorization are in place.

12. Conclusion

This paper presented DeepArchNegotiator, a stakeholder-aware architecture refactoring framework that integrates semantic graph neural modularization with multi-agent requirements negotiation. The key idea is to make architecture impact a first-class negotiation variable. A GNN estimates how requirements and refactoring proposals affect coupling, cohesion, modularity, change impact, migration risk, and behavior risk. Stakeholder agents then negotiate over clinical value, technical feasibility, privacy, operations, regulatory constraints, and product value using guarded Nash bargaining. The result is not merely a cluster assignment or a text summary, but a refactoring-aware agreement: accepted requirements, module-boundary changes, ADRs, traceability, and test obligations. The framework is especially promising for healthcare systems such as the BioArc integration scenario, where clinically valuable features may create unsafe architecture if accepted in the wrong form. DeepArchNegotiator preserves stakeholder value by negotiating the architectural shape of accepted change.

Appendix A. Extended BioArc Guard Rules

Table A.10 defines the BioArc guard rules used in the controlled case. In operational deployments, clinical, legal, and architecture experts refine the rule set before use.

Table A.10: Guard rules for BioArc integration.

Rule ID	Rule
G-PRIV-01	No module outside the Consent Boundary may transfer clinical encounter data without an active consent reference.
G-PRIV-02	Telemedicine services may not directly read or write medication tables; all access must occur through a versioned API or approved event contract.
G-AUD-01	Every cross-system clinical data access must emit an audit event containing user, patient, purpose, timestamp, and consent reference.
G-SAFE-01	Tier 1 life-threatening DDI checks must block prescription signing if a known lethal interaction is detected.
G-SAFE-02	Vendor/API outage may not disable Tier 1 local DDI checks.
G-ARCH-01	Patient identity matching must be performed only by the Patient Identity Gateway.
G-ARCH-02	Shared database access is prohibited unless explicitly approved by an ADR and covered by contract tests.
G-BEH-01	Refactoring may not remove existing prescription, consent, or audit behavior unless the corresponding requirement is reopened and renegotiated.

Appendix B. Reference Repository Structure

Listing 5 gives the reference repository layout for replication artifacts.

Listing 5: Reference repository structure.

```
deep-arch-negotiator/
  data/
    requirements/
    code_snapshots/
    traces/
    constraints/
  src/
    parsers/
    embeddings/
    graph_builder/
    gnn_oracle/
    agents/
    negotiation/
    guards/
    artifacts/
  experiments/
    open_source_systems/
    bioarc_case/
    ablations/
  figures/
  paper/
    DeepArchNegotiator.tex
    deep_arch_negotiator_refs.bib
    generate_deep_arch_figures.py
```

Appendix C. Experimental Configuration and Reproducibility Checklist

The reproducibility checklist below separates the simulated configuration used in this manuscript from metadata that empirical replications must record for each run.

1. Exact codebase versions and commit hashes: recorded for each replication run, including the selected Tomcat and Hadoop benchmark snapshots.

2. Exact requirement scenarios and expert labels: BioArc integration trace, scoring rubric, and expert-label collection protocol.

3. GNN architecture, hidden size, heads, layers, learning rate, and seeds: Heterogeneous GAT, 128 hidden size, 4 heads, 3 layers, LR 1e-4, seeds [42, 43, 44].

4. Embedding models used for code and requirements: CodeBERT (code), PubMedBERT (clinical), ParsBERT (Persian language requirements).

5. Utility weights, disagreement points, and guard thresholds: Disagreement points $d_j = 0.2$, guard threshold $\tau_G = 0.95$.

6. Baseline implementations and parameter settings: DeepModule-compatible configuration and MedNegotiator-compatible agent configuration, with model snapshots logged at run time.

7. Runtime environment, hardware, and model API versions: Ubuntu 22.04, 2x NVIDIA RTX 3090, PyTorch version, and exact model/API snapshots logged at run time.

8. Human evaluation protocol, participant count, consent procedure, and rating rubric: 12-person expert panel, 5-point Likert scale, and ethics/consent metadata recorded before data collection.

9. Statistical tests, effect sizes, and confidence intervals: Wilcoxon signed-rank, significance threshold $\alpha = 0.05$, Cliff's Delta d, and 95% CIs.

10. Data availability and licensing statement: replication package location, persistent identifier, and license recorded upon release.

References

[1] M. M. Lehman, Programs, life cycles, and laws of software evolution, Proceedings of the IEEE 68 (9) (1980) 1060–1076. doi:10.1109/PROC.1980.11805.

[2] W. Cunningham, The wycash portfolio management system, ACM SIGPLAN OOPS Messenger 4 (2) (1992) 29–30. doi:10.1145/157710.157715.

[3] R. C. Seacord, D. Plakosh, G. A. Lewis, Modernizing Legacy Systems: Software Technologies, Engineering Processes, and Business Practices, Addison-Wesley Professional, 2003. doi:10.5555/599767.

[4] V. Tzerpos, R. C. Holt, Acdc: An algorithm for comprehension-driven clustering, in: Proceedings of the Seventh Working Conference on Reverse Engineering, 2000, pp. 258–267. doi:10.1109/WCRE.2000.891477.

[5] O. Maqbool, H. A. Babri, The weighted combined algorithm: A linkage algorithm for software clustering, in: Eighth Euromicro Working Conference on Software Maintenance and Reengineering, 2004, pp. 15–24. doi:10.1109/CSMR.2004.1281402.

[6] M. Harman, S. A. Mansouri, Y. Zhang, Search-based software engineering: Trends, techniques and applications, ACM Computing Surveys 45 (1) (2012) 1–61. doi:10.1145/2379776.2379787.

[7] B. S. Mitchell, S. Mancoridis, On the automatic modularization of software systems using the bunch tool, IEEE Transactions on Software Engineering 32 (3) (2006) 193–208. doi:10.1109/TSE.2006.31.

[8] K. Praditwong, M. Harman, X. Yao, Software module clustering as a multi-objective search problem, IEEE Transactions on Software Engineering 37 (2) (2011) 264–282. doi:10.1109/TSE.2010.14.

[9] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, arXiv preprint arXiv:1710.10903 (2018). doi:10.48550/arXiv.1710.10903.

[10] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, M. Zhou, Codebert: A pre-trained model for programming and natural languages, in: Findings of the Association for Computational Linguistics: EMNLP 2020, 2020, pp. 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139.

[11] D. Guo, S. Ren, S. Lu, Z. Feng, D. Tang, S. Liu, L. Zhou, N. Duan, A. Svyatkovskiy, S. Fu, M. Tufano, S. K. Deng, C. Clement, D. Drain, N. Sundaresan, J. Yin, D. Jiang, M. Zhou, Graphcodebert: Pre-training code representations with data flow, arXiv preprint arXiv:2009.08366 (2020). doi:10.48550/arXiv.2009.08366.

[12] J. Zhou, G. Cui, S. Hu, Z. Zhang, C. Yang, Z. Liu, L. Wang, C. Li, M. Sun, Graph neural networks: A review of methods and applications, AI Open 1 (2020) 57–81. doi:10.1016/j.aiopen.2021.01.001.

[13] M. Tanhaei, R. Alizadehsani, Deepmodule: Learning to refactor software architectures via semantic-aware graph neural networks, user-provided manuscript, version V2 (2026).

[14] M. Tanhaei, Mednegotiator: Automating requirements negotiation in healthcare; simulating clinical and technical perspectives using generative ai agents, Array 30 (2026) 100783. doi:10.1016/j.array.2026.100783.

[15] I. Sommerville, Integrated requirements engineering: A tutorial, IEEE Software 22 (1) (2005) 16–23. doi:10.1109/MS.2005.13.

[16] B. H. C. Cheng, J. M. Atlee, Research directions in requirements engineering, in: Future of Software Engineering (FOSE 2007), 2007, pp. 285–303. doi:10.1109/FOSE.2007.17.

[17] O. C. Z. Gotel, A. C. W. Finkelstein, An analysis of the requirements traceability problem, in: Proceedings of the First International Conference on Requirements Engineering, 1994, pp. 94–101. doi:10.1109/ICRE.1994.292398.

[18] H. W. J. Rittel, M. M. Webber, Dilemmas in a general theory of planning, Policy Sciences 4 (2) (1973) 155–169. doi:10.1007/BF01405730.

[19] T. Materla, E. A. Cudney, J. Antony, The application of kano model in the healthcare industry: A systematic literature review, Total Quality Management & Business Excellence 30 (5–6) (2019) 660–681. doi:10.1080/14783363.2017.1328980.

[20] K. Matzler, H. H. Hinterhuber, F. Bailom, E. Sauerwein, How to delight your customers, Journal of Product & Brand Management 5 (2) (1996) 6–18. doi:10.1108/10610429610119469.

[21] J. F. Nash, The bargaining problem, Econometrica 18 (2) (1950) 155–162. doi:10.2307/1907266.

[22] A. Rubinstein, Perfect equilibrium in a bargaining model, Econometrica 50 (1) (1982) 97–109. doi:10.2307/1912531.

[23] S. S. Fatima, S. Kraus, M. Wooldridge, Principles of Automated Negotiation, Cambridge University Press, 2014. doi:10.1017/CBO9780511751691.

[24] P. Faratin, C. Sierra, N. R. Jennings, Negotiation decision functions for autonomous agents, Robotics and Autonomous Systems 24 (3–4) (1998) 159–182. doi:10.1016/S0921-8890(98)00029-3.

[25] I. Rahwan, S. D. Ramchurn, N. R. Jennings, P. McBurney, S. Parsons, L. Sonenberg, Argumentation-based negotiation, The Knowledge Engineering Review 18 (4) (2003) 343–375. doi:10.1017/S0269888904000098.

[26] S. Parsons, C. Sierra, N. R. Jennings, Agents that reason and negotiate by arguing, Journal of Logic and Computation 8 (3) (1998) 261–292. doi:10.1093/logcom/8.3.261.

[27] L. C. Briand, J. W. Daly, J. K. Wust, A unified framework for coupling measurement in object-oriented systems, IEEE Transactions on Software Engineering 25 (1) (1999) 91–121. doi:10.1109/32.748920.

[28] G. Bavota, A. De Lucia, A. Marcus, R. Oliveto, Automating extract class refactoring: An improved method and its evaluation, Empirical Software Engineering 19 (2014) 1617–1664. doi:10.1007/s10664-013-9256-x.

[29] C. Abid, V. Alizadeh, M. Kessentini, T. d. N. Ferreira, D. Dig, 30 years of software refactoring research: A systematic literature review, arXiv preprint arXiv:2007.02194 (2020). doi:10.48550/arXiv.2007.02194.

[30] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, arXiv preprint arXiv:1706.02216 (2017). doi:10.48550/arXiv.1706.02216.

[31] E. A. Leicht, M. E. J. Newman, Community structure in directed networks, Physical Review Letters 100 (2008) 118703. doi:10.1103/PhysRevLett.100.118703.

[32] U. Alon, M. Zilberstein, O. Levy, E. Yahav, code2vec: Learning distributed representations of code, Proceedings of the ACM on Programming Languages 3 (POPL) (2019) 1–29. doi:10.1145/3290353.

[33] B. Roziere, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Defossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, G. Synnaeve, Code llama: Open foundation models for code, arXiv preprint arXiv:2308.12950 (2023). doi:10.48550/arXiv.2308.12950.

[34] R. Li, L. B. Allal, Y. Zi, N. Muennighoff, D. Kocetkov, C. Mou, M. Marone, C. Akiki, J. Li, J. Chim, Q. Liu, E. Zheltonozhskii, T. Y. Zhuo, T. Wang, O. Dehaene, B. Davaadorj, J. Lamy-Poirier, J. Monteiro, O. Shliazhko, N. Gontier, N. Meade, M.-H. Zhu, B. Lipkin, D. Bahdanau, Y. Li, A. Chim, Q. Liu, S. Iyer, J. Li, et al., Starcoder: May the source be with you!, arXiv preprint arXiv:2305.06161 (2023). doi:10.48550/arXiv.2305.06161.

[35] A. Lozhkov, L. B. Allal, L. von Werra, T. Wolf, et al., Starcoder 2 and the stack v2: The next generation, arXiv preprint arXiv:2402.19173 (2024). doi:10.48550/arXiv.2402.19173.

[36] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, Large language models for software engineering: A systematic literature review, ACM Transactions on Software Engineering and Methodology 33 (8) (2024) 1–77. doi:10.1145/3695988.

[37] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, C. Wang, Autogen: Enabling next-gen llm applications via multi-agent conversation, arXiv preprint arXiv:2308.08155 (2023). doi:10.48550/arXiv.2308.08155.

[38] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, Y. Cao, React: Synergizing reasoning and acting in language models, arXiv preprint arXiv:2210.03629 (2023). doi:10.48550/arXiv.2210.03629.

[39] S. Es, J. James, L. Espinosa-Anke, S. Schockaert, Ragas: Automated evaluation of retrieval augmented generation, in: Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations, 2024, pp. 150–158. doi:10.18653/v1/2024.eacl-demo.16.

[40] BioArc, Bioarc: Smart hospital information system, https://bioarc.ir/landing/index?lang=en, accessed as a public product description; case-study details in this manuscript are marked as TBD where empirical values are not yet measured (2024).

[41] IEEE Standards Association, Ieee recommended practice for software requirements specifications, IEEE Std 830-1998 (1998). doi:10.1109/IEEESTD.1998.88286.

[42] C. Wohlin, P. Runeson, M. Host, M. C. Ohlsson, B. Regnell, A. Wesslen, Experimentation in Software Engineering, Springer, 2012. doi:10.1007/978-3-642-29044-2.